Detailed 10-Day React + TypeScript Training Guide

Designed for an experienced .NET + KnockoutJS Developer

Program Objectives

At the end of this two-week program the developer should be able to independently develop enterprise-grade React applications using TypeScript, integrate REST APIs, build reusable components, follow modern frontend architecture, understand React Hooks, optimize performance, write maintainable code, participate in code reviews and contribute to production projects.

Day 1 - Modern JavaScript (ES6+)

Objective: Understand modern JavaScript required by React.

Topics

• let/const

• Arrow Functions

• Template Literals

• Destructuring

• Spread/Rest

• Array Methods

• Promises

• Async/Await

• Modules

Hands-on Assignment

• Create Employee model

• Employee CRUD using arrays

• Filtering & Sorting

• Mock async API

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 2 - TypeScript

Objective: Learn strong typing used in enterprise React.

Topics

• Primitive Types

• Interfaces

• Types

• Enums

• Generics

• Utility Types

• Optional Properties

• Type Narrowing

• tsconfig

Hands-on Assignment

• Convert Day 1 to TS

• Create reusable interfaces

• Generic API Response

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 3 - React Fundamentals

Objective: Learn component-based development.

Topics

• JSX

• Functional Components

• Props

• Component Composition

• Project Structure

• Rendering

Hands-on Assignment

• Employee Card

• Department Card

• Dashboard

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 4 - State Management Basics

Objective: Master interactive UI.

Topics

• useState

• Events

• Conditional Rendering

• Lists

• Keys

• Lifting State

Hands-on Assignment

• Employee CRUD

• Search

• Department Filter

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 5 - Hooks & API

Objective: Connect frontend with backend.

Topics

• useEffect

• Axios

• Fetch

• Loading

• Error Handling

• Environment Variables

Hands-on Assignment

• Consume DummyJSON

• Pagination

• Search

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 6 - Routing & Forms

Objective: Build multi-page applications.

Topics

• React Router

• Nested Routes

• Route Params

• React Hook Form

• Validation

Hands-on Assignment

• HR Portal

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 7 - Reusable Components

Objective: Enterprise UI architecture.

Topics

• Button

• Input

• Table

• Modal

• Loader

• Pagination

• Custom Hooks

Hands-on Assignment

• Build reusable library

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 8 - Advanced React

Objective: Improve performance.

Topics

• Context API

• Reducers

• React.memo

• useMemo

• useCallback

• Lazy Loading

• Code Splitting

• Error Boundaries

Hands-on Assignment

• Optimize previous application

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 9 - Production Development

Objective: Apply concepts to a real module.

Topics

• Git Workflow

• Code Reviews

• Folder Structure

• API Integration

• Coding Standards

Hands-on Assignment

• Develop one production module independently

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Day 10 - Assessment

Objective: Validate readiness.

Topics

• Architecture

• Coding

• Debugging

• Review

• Refactoring

Hands-on Assignment

• Complete Employee Management Portal

Expected Outcome

The developer should demonstrate the ability to apply the above concepts without assistance, explain design decisions, write clean and reusable code, and submit changes through Git following coding standards.

Final Hands-on Exercises

Exercise 1 – Task Management System

Develop a complete Task Management application using React + TypeScript with login screen (dummy), dashboard, CRUD operations, search, sorting, filtering, pagination, reusable components, Context API, protected routes and responsive UI.

Exercise 2 – Product Catalog

Create an e-commerce product catalog integrating a REST API. Implement categories, search, product details, cart, reusable hooks, loading states, error handling and performance optimization.

Exercise 3 – KnockoutJS Migration Challenge

Take an existing KnockoutJS screen from your current application and migrate it to React + TypeScript. Preserve functionality while improving architecture, component reusability, folder structure and maintainability.

Evaluation Criteria

• JavaScript & TypeScript Fundamentals – 15%

• React Components & Hooks – 20%

• State Management – 10%

• API Integration – 10%

• Routing & Forms – 10%

• Reusable Components – 10%

• Performance Optimization – 10%

• Code Quality & Best Practices – 10%

• Problem Solving & Debugging – 5%

Recommended Learning Resources

• Official React Documentation

• TypeScript Handbook

• React Router Documentation

• React Hook Form Documentation

• Axios Documentation

• ESLint & Prettier